



UQ24CA342A Software Testing and Automation

Mrs. Sowmya BP

Department of Computer Applications

UQ24CA342A

Software Testing and Automation

Testing Fundamentals

Mrs. Sowmya BP

Department of Computer Applications

Software Testing is the process of evaluating a software application or system to identify any defects, ensure it meets the specified requirements, and verify that it functions as expected.

Why is Software Testing Important?

- **Error Detection** – Identifies bugs and defects early.
- **Quality Assurance** – Ensures the software is reliable, secure, and performs well.
- **Cost-Effective** – Fixing bugs early is cheaper than post-deployment.
- **Customer Satisfaction** – Helps in delivering a product that meets user expectations.
- **Compliance** – Verifies that the software meets industry standards and regulations.

UQ24CA342A Software Testing and Automation

Introduction to Software Testing

- Examples of Testing in Daily Life

Situation	What You're Testing	Expected Result
Trying your house key in the lock	Does the key unlock the door?	Door opens
Tasting food before serving	Is the food too salty/spicy?	Balanced taste
Checking phone before a call	Is the phone charged and has signal?	Call connects successfully
Switching on the fan/light	Does it turn on properly?	Appliance starts working
Wearing a shoe before buying	Is it comfortable and the right size?	Comfortable fit
Sending a test email to yourself	Do the email service and formatting work?	Email is received and displayed correctly
Checking ATM withdrawal	Can I withdraw money with this card and PIN?	Cash is dispensed
Clicking "Send" on a WhatsApp message	Is the message being sent and delivered?	Message shows blue ticks

UQ24CA342A Software Testing and Automation

Basics of Software Testing

Define the following

- Error
- Fault
- Failure
- Test
- Test Case
- Test Plan
- Test Script
- Test Suite
- Test Scenario
- Test Data
- Bug/Defect
- Test Environment

UQ24CA342A Software Testing and Automation

Keywords

Keyword	Definition	Example
Error (also called Mistake)	A human action that produces an incorrect result in the software.	A developer writes $x = y + z$ instead of $x = y - z$.
Fault (also called Defect or Bug)	A flaw in the software caused by an error. It's something wrong in the code or design.	The incorrect line $x = y + z$ remains in the code.
Failure	The software does not behave as expected during execution due to a fault.	The app crashes or gives wrong output when x is calculated.
Test	The act of executing software to find defects.	Running a test to see if login works correctly.
Test Case	A specific set of inputs and expected results to validate a feature.	Entering a valid username and password to check login success.
Test Plan	A document describing the scope, approach, and schedule of testing.	A plan for testing a new mobile app release.
Test Script	A written or automated script that performs a test.	A Python script using Selenium to test form submission.

UQ24CA342A Software Testing and Automation

Keywords

Keyword	Meaning
Test Suite	A collection of test cases intended to be executed together.
Test Scenario	A high-level description of what to test, often covering an end-to-end functionality.
Test Data	Data used to execute test cases.
Bug/Defect	A flaw or error in software that causes it to behave unexpectedly.
Test Environment	Setup of hardware, software, and network for testing.
Pass	Test case executed successfully and result was as expected.
Fail	Test case failed due to a defect or mismatch with expected result.

UQ24CA342A Software Testing and Automation

Basics of Software Testing

Term	Definition	Calculator Example
Error	A human mistake in coding or design.	Developer accidentally writes + instead of - for subtraction.
Fault	A problem in the code caused by an error.	The code wrongly adds numbers in the subtraction function.
Failure	When the software does not behave as expected during execution.	You press 5 - 2, but the calculator shows 7 — incorrect result.
Test	The process of executing the application to find defects.	Running the calculator app and checking if $3 + 4 = 7$.
Test Case	A set of inputs, conditions, and expected results for testing.	Input: $2 + 3 \rightarrow$ Expected Output: 5
Test Plan	A document outlining testing strategy, scope, schedule, and resources.	Document that says: “We will test addition, subtraction, division, multiplication. Priority to division bugs.”
Test Script	A written or automated sequence of steps to perform a test.	A script that automatically enters numbers and checks if the output is correct.

UQ24CA342A Software Testing and Automation

Basics of Software Testing

Test Suite	A collection of related test cases.	A group of test cases for all calculator functions: add, subtract, multiply, divide.
Test Scenario	A high-level idea of what to test.	“Test basic arithmetic operations on calculator.”
Test Data	Input data used in test cases.	Numbers like 5, 0, -3, 100 used to test calculations.
Bug / Defect	A mismatch between expected and actual behavior.	Subtraction of 5 - 3 returns 9 → clearly a bug.
Test Environment	The setup where testing is done (hardware, software, OS).	Calculator app installed on Android 11 with 4GB RAM phone.
Pass	Test case executed and result matched expected output.	$6 \times 3 = 18$ → correct → test passed.
Fail	Test case executed and result did not match expected output.	$6 \div 3$ returns 0 instead of 2 → test failed.

UQ24CA342A Software Testing and Automation

Determining the Scope of Testing



Scope of testing involves identifying the testing areas, features, and functionality that need to be tested, as well as defining the testing boundaries.

- Ex:Mobile Banking App

Features to test (in-scope):	Features not to test (out-of-scope):
User login and logout	Admin panel access
Balance check	Backend database performance
Fund transfer	SMS delivery (handled by third-party provider)
Transaction history	database performance

UQ24CA342A Software Testing and Automation

Determining the Scope of Testing



Determining the Scope of Testing based on various factors such as the i) application's complexity, ii) the risk associated with the application, and iii) the testing resources available.

Ex: Mobile Banking App

- **Complexity:** Fund transfer involves multiple flows (IMPS, NEFT, UPI) and is highly complex.
- **Risk:** Mistakes in fund transfer may lead to financial loss. So, this is high-risk and must be thoroughly tested.
- **Resources:** Limited testers and time → focus on critical user flows only like login, transfer, and transaction confirmation.

UQ24CA342A Software Testing and Automation

Determining the Scope of Testing



The scope of testing should be reviewed and updated regularly during the testing process to ensure that the testing effort is on track, and all the testing objectives are being met.

Ex: Mobile Banking App

- A new UPI feature was added
- There were frequent changes to UI

Scenario

A team is developing an Online Food Delivery Application. As a software tester, identify which features should be included in the testing scope (in-scope) and which should be excluded (out-of-scope) for the current release.

UQ24CA342A Software Testing and Automation

Classroom Activity: Identify the Scope of Testing



Answer

Features to Test (In-Scope)	Features Not to Test (Out-of-Scope)
User registration and login	Restaurant's internal order management system
Search restaurants and food items	Restaurant kitchen operations
Add/remove items from cart	Third-party payment gateway internal processing
Apply coupon codes	GPS satellite accuracy
Place food order	Delivery partner's payroll system
Track order status	Mobile network performance
View order history	Cloud server infrastructure managed by provider
Cancel order before preparation	SMS/Email service provider infrastructure

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the primary objective of software testing?

- A) Develop software quickly
- B) Find defects and verify requirements
- C) Increase code length
- D) Reduce programming languages

2. Which feature is out-of-scope in the Mobile Banking example?

- A) User Login
- B) Fund Transfer
- C) Transaction History
- D) Admin Panel Access

3. A banking application has only two testers and one week before release. Which feature should be tested first?

- A) Background theme
- B) Login and Fund Transfer
- C) Font color
- D) Help documentation

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. D
3. B

Selecting the approach taken will depend on a number of factors, such as the complexity of the software, the project timeline, and the available resources.

Most common testing approaches are:

- Black-box testing
- White-box testing
- Grey-box testing
- Manual testing
- Automated testing
- Exploratory testing
- Regression testing

UQ24CA342A Software Testing and Automation

Selecting the Testing Approach

Black-box testing:

- This approach tests the functionality of the software without any knowledge of the internal workings of the software.
- Testers use requirements specifications to create test cases.

Ex: Withdrawing Cash from an ATM

You go to an ATM to withdraw ₹500. As a user, you:

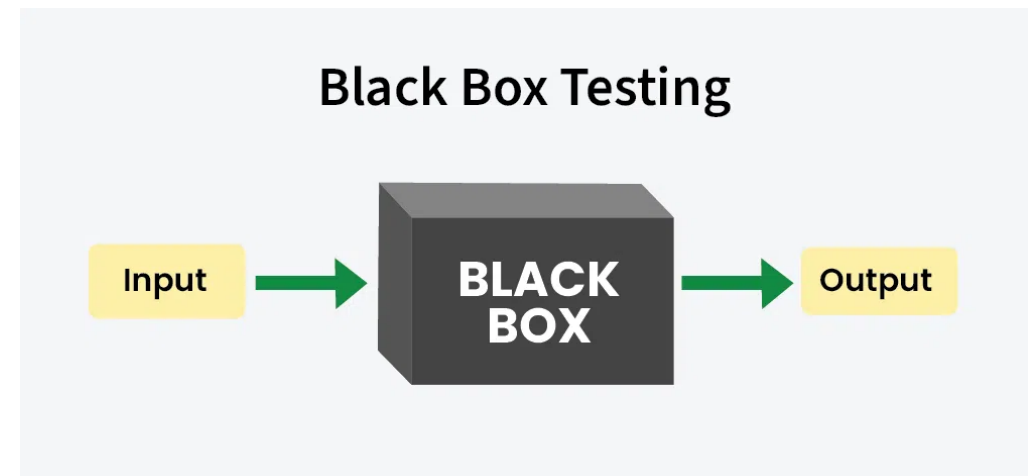
Insert your card

Enter your PIN

Choose “Withdraw Money”

Enter ₹500

Press “Confirm”



UQ24CA342A Software Testing and Automation

Selecting the Testing Approach



Black-box testing:

What You're Testing (as a user)

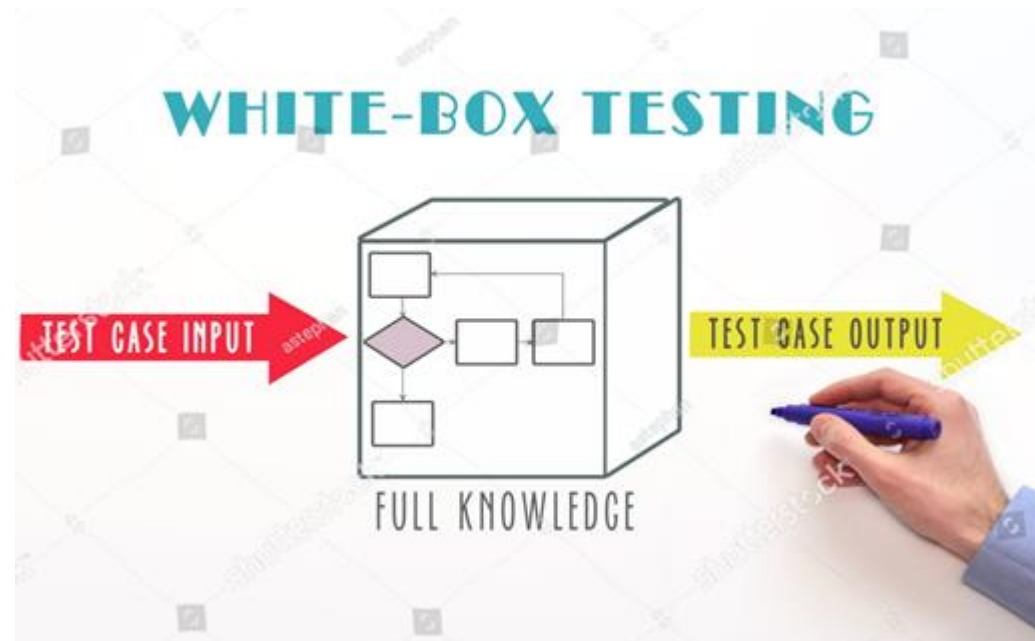
Input	Expected Output
Correct PIN, ₹500 in account	₹500 is dispensed, receipt is printed
Incorrect PIN	Error: "Invalid PIN. Try again."
Insufficient balance	Error: "Insufficient Funds"
Card expired	Error: "Card Expired"

UQ24CA342A Software Testing and Automation

Selecting the Testing Approach

White-box testing:

- This approach tests the internal workings of the software, such as code, logic, and algorithms.
- Testers use programming skills to create test cases.



UQ24CA342A Software Testing and Automation

Selecting the Testing Approach



Ex: Developer testing the PIN verification function inside an ATM system.

```
def verify_pin(entered_pin, actual_pin):  
    if len(entered_pin) != 4:  
        return "Invalid length"  
    if entered_pin == actual_pin:  
        return "PIN Verified"  
    else:  
        return "Incorrect PIN"
```

Test Case ID	Input (entered_pin, actual_pin)	Expected Output
TC_01	"123", "1234"	Invalid length
TC_02	"1234", "1234"	PIN Verified
TC_03	"1111", "1234"	Incorrect PIN

Gray-box testing:

- This approach combines the elements of black-box and white-box testing.
- Testers have limited knowledge of the internal workings of the software, allowing them to test both the functionality and the internal workings of the software.



Ex: Testing the **checkout process** of an e-commerce site like Amazon or Flipkart.

Test Scenario	Input	What You Test
Apply a valid coupon	Cart amount: ₹1000, coupon:10%	Check UI response AND verify backend API returns 10% discount
Place an order and verify status in DB	Order placed successfully	Test that the UI shows confirmation AND DB shows correct status = "Paid"
Simulate session expiry during checkout	Inactivity for 10 mins	Check if session token is invalidated via API call
Review logs for failed payment	Payment fails via UPI	Use log files or monitoring tools to trace internal error codes

UQ24CA342A Software Testing and Automation

Selecting the Testing Approach

Manual testing:

- This approach involves manual execution of test cases.
- Testers execute the test cases by hand and record the results.



UQ24CA342A Software Testing and Automation

Selecting the Testing Approach

Place Food Order

Manual Test Cases Format

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_001	Verify login with valid credentials	1. Open app 2. Enter username & password 3. Tap "Login"	Username: testuser Password: 123456	Dashboard/homepage should be displayed		Pass/Fail
TC_002	Search for a food item	1. Tap on Search 2. Type "Pizza" 3. Press Enter/Search icon	Pizza	List of restaurants with pizza should appear		Pass/Fail
TC_003	Add item to cart	1. Select a restaurant 2. Choose a pizza 3. Tap "Add to Cart"	Any pizza	Item is added to the cart with updated price		Pass/Fail
TC_004	Proceed to checkout	1. Open cart 2. Tap "Proceed to Checkout"	Cart contains 1 item	Checkout page is displayed		Pass/Fail

Automated testing:

- This approach involves the use of software tools to execute test cases automatically.
- This is a faster and more efficient testing approach than manual testing.

Few Tools
Selenium
Cypress
Playwright
Appium
Robot Framework

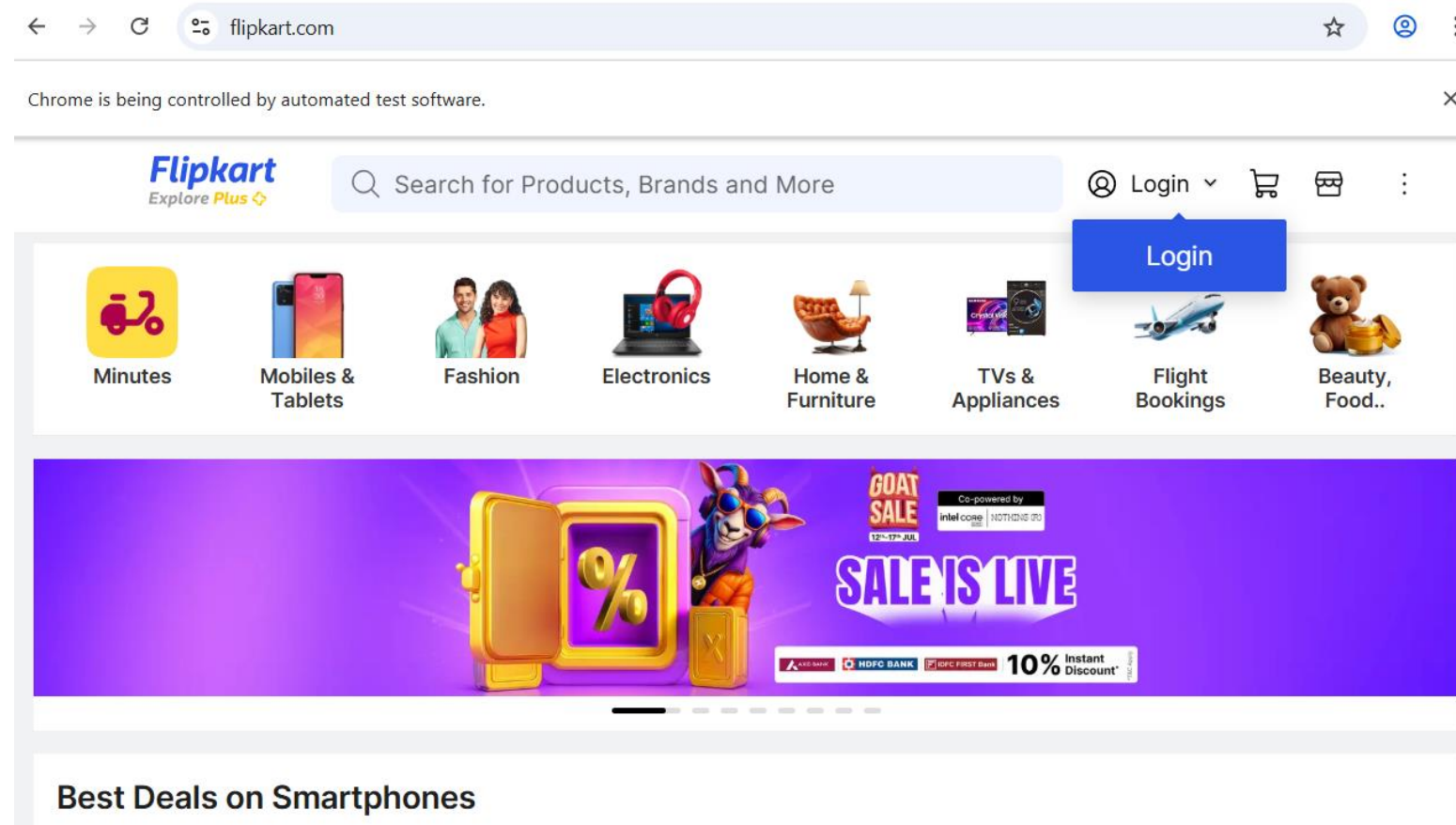


UQ24CA342A Software Testing and Automation

Selecting the Testing Approach

Example: Automating the process of

- Opening the website
- Logging in
- Searching for a product
- Adding it to the cart
- Verifying the cart



Exploratory testing:

- This approach involves testing the software without a formal test plan.
- Testers use their knowledge of the software to identify defects.

Example: New Chat Feature in a Messaging App (e.g., WhatsApp, Telegram)

Activity	What You Explore
Open chat filter settings	Check if the UI loads correctly
Try different filter options (unread, media, groups)	See how chat list changes and behaves
Send/receive messages during filtering	Check if filtered chats update in real-time
Combine filters randomly	Discover edge cases (e.g., “Unread + Groups”)
Use the app on light/dark mode	UI behavior and compatibility
Check reactions to unknown contacts	Does the filter include/exclude them?
Look for crashes or glitches	Unexpected results or UI errors

Regression testing:

- This approach involves retesting the software after a change has been made.
- The purpose of regression testing is to ensure that the change did not introduce any new defects.

UQ24CA342A Software Testing and Automation

Selecting the Testing Approach

Example: Messaging App – New Emoji Reactions

You add emoji reactions to messages in a chat app should not affect on other functionalities.

- **Regression Tests:**
- Sending and receiving text messages
- Group chat functionality
- Push notifications still work
- Chat deletion and media sharing still function properly



UQ24CA342A Software Testing and Automation

Classroom Quiz

1. Exploratory Testing is mainly performed:

- A) Using predefined scripts only
- B) Without a formal test plan
- C) Only using automation tools
- D) Only after release

2. Which statement best describes Automated Testing?

- A) It completely replaces manual testing.
- B) It is mainly useful for repetitive test execution using automation tools.
- C) It does not require test cases.
- D) It is only used for performance testing.

3. A developer tests every condition and branch inside a PIN verification function. Which testing approach is this?

- A) Black-box Testing
- B) White-box Testing
- C) Manual Testing
- D) Acceptance Testing

4. Gray-box testing combines the features of:

- A) Manual & Automation Testing
- B) Functional & Performance Testing
- C) Black-box & White-box Testing
- D) Unit & Integration Testing

5. Which testing approach executes test cases without automation tools?

- A) Manual Testing
- B) Automated Testing
- C) Regression Testing
- D) Gray-box Testing

6. Which testing approach checks software functionality without looking at the source code?

- A) White-box Testing
- B) Black-box Testing
- C) Gray-box Testing
- D) Unit Testing

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B

2. B

3. B

4. C

5. A

6. B

In software testing, testing resources are all the assets—human, technical, and material—needed to plan, execute, and manage testing activities.

The testing resources that may be required include the following:

- Personnel
- Testing tools
- Infrastructure
- Data
- Budget

Personnel: This includes the testing team members, test leads, and test managers who will be responsible for performing the testing activities. Ex:

- Testers & QA engineers (manual and automation)
- Developers (for unit testing and fixing defects)
- Test managers / leads
- Domain experts (to ensure business logic is correct)
- Stakeholders (for acceptance testing)

UQ24CA342A Software Testing and Automation

Identifying Testing Resources

Personnel

This includes the testing team members, test leads, and test managers who will be responsible for performing the testing activities.



Testers & QA Engineers
(Manual & Automation)

Design, execute, and report test cases to find defects and ensure quality.



Developers

Perform unit testing and fix defects to ensure the application is stable and meets requirements.



Test Managers / Leads

Plan, monitor, and manage the testing activities and ensure the team meets the quality goals.



Domain Experts

Provide deep knowledge of the business domain to ensure that the system meets business needs and rules.



Stakeholders

Review test results and provide acceptance to ensure the solution meets business requirements and user expectations.

Testing tools: The testing team may require various testing tools such as test management tools, defect tracking tools, automation tools, performance testing tools, and security testing tools to help perform the testing activities.

Ex:

- Testing tools (e.g., Selenium, JUnit, PyTest, Postman)
- Test management tools (e.g., Jira, TestRail)
- Automation frameworks

UQ24CA342A Software Testing and Automation

Identifying Testing Resources

TESTING TOOLS

The testing team may require various testing tools such as test management tools, defect tracking tools, automation tools, performance testing tools, and security testing tools to help perform the testing activities.

Examples:



Testing tools

(e.g., Selenium, JUnit, PyTest, Postman)



Test management tools

(e.g., Jira, TestRail)



Automation frameworks

1. TESTING TOOLS



Selenium



JUnit



pytest



Postman

Used for functional, API, unit and integration testing.

2. TEST MANAGEMENT TOOLS



Jira



TestRail



Zephyr

Used for test case management, planning, execution tracking and reporting.

3. AUTOMATION FRAMEWORKS



TestNG



cypress



Robot
Framework

Provide structure and reusability to create and maintain automated tests.

4. PERFORMANCE TESTING TOOLS



JMeter



LoadRunner



k6

Used to test application performance, load, stress, scalability and stability.

5. SECURITY TESTING TOOLS



OWASP ZAP



Burp Suite



Nessus



Wireshark

Used to identify vulnerabilities, security gaps and ensure application security.

BENEFITS

- ✓ Improves test efficiency
- ✓ Enhances accuracy
- ✓ Better collaboration
- ✓ Reduces time to market
- ✓ Ensures quality and security.

Infrastructure: The testing team may require hardware and software infrastructure such as servers, network equipment, and testing environments.

Ex:

- Test servers, staging environments, cloud instances
- Required devices (PCs, mobile phones, tablets)
- Network configurations for performance/security testing

UQ24CA342A Software Testing and Automation

Identifying Testing Resources

INFRASTRUCTURE

The testing team may require hardware and software infrastructure such as servers, network equipment, and testing environments.

Examples:



Test servers, staging environments, cloud instances

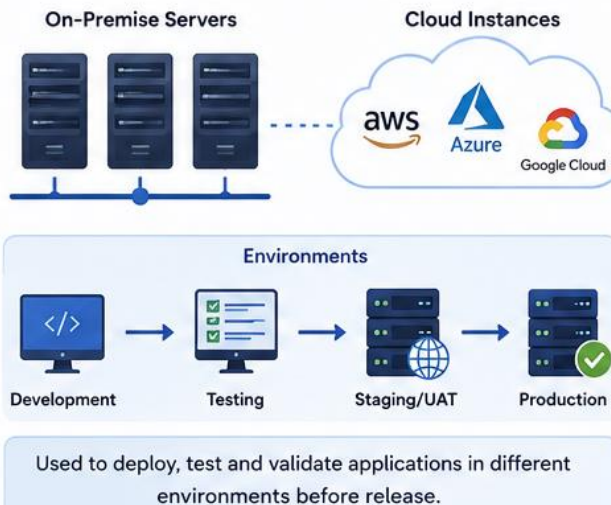


Required devices (PCs, mobile phones, tablets)



Network configurations for performance/security testing

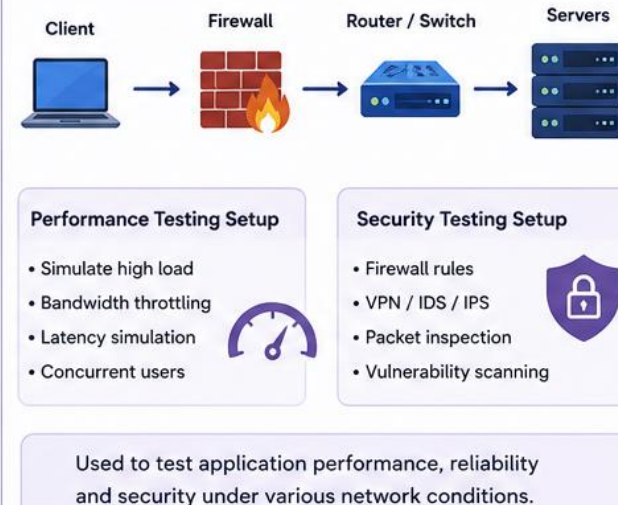
1. TEST SERVERS & ENVIRONMENTS



2. REQUIRED DEVICES



3. NETWORK CONFIGURATIONS



Data: The testing team may require data that represents the actual production environment to perform the testing activities. Ex:

- Customer data
- Product data
- Transaction data

UQ24CA342A Software Testing and Automation

Identifying Testing Resources



The testing team may require data that represents the actual production environment to perform the testing activities.

Examples:

1. CUSTOMER DATA



Data related to customers/users of the application.

Customer ID	Name	Email	Phone	Location
CUST001	Rahul Sharma	rahul@example.com	9876543210	Bangalore
CUST002	Priya Nair	priya@example.com	9123456780	Mumbai
CUST003	Amit Verma	amit@example.com	9988776655	Delhi
...	...	...	...	...



Used to test registration, login, profile, preferences, and customer-related features.

2. PRODUCT DATA



Data related to products/services available in the application.

Product ID	Product Name	Category	Price	Stock
P1001	Smartphone	Electronics	25000	120
P1002	Laptop	Electronics	55000	80
P1003	Headphones	Accessories	1500	250
...	...	...	...	...



Used to test product search, details, inventory, pricing, and ordering flows.

3. TRANSACTION DATA



Data related to transactions/orders performed in the system.

Txn ID	Customer ID	Product ID	Amount	Status	Date
TXN1001	CUST001	P1001	25000	Success	10-05-2025
TXN1002	CUST002	P1002	55000	Success	11-05-2025
TXN1003	CUST003	P1003	1500	Failed	11-05-2025
...	...	...	...	...	...



Used to test order placement, payments, refunds, cancellations, and reports.

Budget: The testing team may require a budget to cover the costs of the testing activities. This may include costs for personnel, testing tools, infrastructure, and data. Ex:

- Funds allocated for tools, training, and infrastructure
- Time allocated for each testing phase

UQ24CA342A Software Testing and Automation

Identifying Testing Resources



BUDGET



The testing team may require a budget to cover the costs of the testing activities. This may include costs for personnel, testing tools, infrastructure, and data.

Examples



Funds allocated for tools, training, and infrastructure



Time allocated for each testing phase

1. BUDGET BREAKDOWN (Example)

Budget Category	Description	Estimated Cost (INR)	% of Total Budget
 Personnel	Salaries for testers, test leads, and test managers	₹ 6,00,000	40%
 Testing Tools	Licenses for test management, automation, performance & security tools	₹ 3,00,000	20%
 Infrastructure	Servers, cloud instances, devices, network setup	₹ 3,50,000	23%
 Test Data	Data preparation, masking, data refresh	₹ 1,00,000	7%
 Training	Training & certification for testing team	₹ 50,000	3%
 Contingency	Unforeseen expenses	₹ 50,000	3%
TOTAL BUDGET		₹ 15,00,000	100%

2. TIME ALLOCATION FOR TESTING PHASES (Example)

Testing Phase	Activities	Estimated Time	% of Total Time
 Test Planning	Test strategy, scope, resource planning, test plan	1 Week	10%
 Test Design	Test case creation, test data preparation	2 Weeks	20%
 Test Execution	Test case execution (Manual & Automated)	5 Weeks	40%
 Defect Management	Logging, tracking, re-testing, verification	3 Weeks	20%
 Test Closure	Test summary report, metrics, lessons learned	1 Week	10%
TOTAL TIME		12 Weeks	100%

Real-Time Example: Mobile Banking App Testing

Personnel	Manual tester, security tester, automation testers
Testing tools	Selenium, JIRA, Postman, Burp Suite
Infrastructure	• Real devices (Android/iOS) • Virtual testing platforms (e.g., BrowserStack) • Test servers for staging the banking app • VPN or secure test environments for confidential data access
Data	• Test users with different credit scores • Dummy account balances • Simulated PAN/Aadhar numbers • Edge case data like blank forms, invalid loan amounts
Budget	Salaries for testers and tools licenses

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. Why should test data closely resemble production data?

- A) To increase programming speed
- B) To produce realistic and reliable testing results
- C) To reduce the number of developers
- D) To avoid writing test cases

2. Which type of test data is used to verify order placement, payments, and refunds?

- A) Customer Data
- B) Product Data
- C) Transaction Data
- D) Employee Data

3. Why is proper time allocation important during software testing?

- A) It reduces the number of software features.
- B) It ensures every testing phase is completed effectively and on schedule.
- C) It eliminates the need for testers.
- D) It replaces automation tools.

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. C
3. B

A test schedule is a detailed plan that outlines the testing activities, timelines, and resources required to carry out the testing activities.

The following are the steps to develop a test schedule:

- Determine the testing timeline
- Identify the testing activities
- Determine the order of testing activities
- Allocate resources
- Develop the test schedule
- Review and refine
- Communicate the test schedule

UQ24CA342A Software Testing and Automation

Developing the Test Schedule



Determine the testing timeline

- The testing timeline is the period in which the testing activities will take place.
- It is important to consider the project timeline, including the delivery date, milestones, and other important events.

Identify the testing

- Activities required for each stage of the SDLC such as functional testing, performance testing, security testing, and so on.

Determine the order of testing activities

- Determine the order in which the testing activities will be carried out.
- This may depend on the type of testing, the requirements of the project, and the interdependencies of the various testing activities.

Allocate resources:

- Allocate resources to each testing activity.
- This includes personnel, tools, hardware, and software required for each testing activity.

Develop the test schedule:

- Using the information collected, create a detailed schedule of the testing activities.
- The schedule should include the start and end dates of each testing activity,
- The resources allocation to each activity, and the dependencies between the various testing activities.

UQ24CA342A Software Testing and Automation

Developing the Test Schedule



Review and refine:

- Once the test schedule is created, it should be reviewed and refined to ensure that it is accurate and realistic.
- The review should also include input from key stakeholders to ensure that the test schedule aligns with the project objectives and priorities.

Communicate the test schedule:

- The final step is to communicate the test schedule to all stakeholders, including the development team, project manager, and other relevant parties.
- It is important to ensure that all stakeholders understand the test schedule and their roles and responsibilities in carrying out the testing activities as per the schedule.

UQ24CA342A Software Testing and Automation

Developing the Test Schedule



Ex: Online food delivery app

1. Determine the testing timeline

Goal: Decide the overall testing start and end dates.

Example: The development team plans to deliver the first testable build on **Sept 1**.

Final release is **Oct 15**.

Testing Timeline: Sept 1 – Oct 10 (5 weeks).

Ex: Online food delivery app

2. Identify the testing activities

Unit testing (done by developers)

Integration testing (API + database + payment gateway)

UI/UX testing (mobile app responsiveness)

Functional testing (placing orders, tracking delivery)

Performance testing (load tests for 10k concurrent users)

Security testing (payment & personal data)

User acceptance testing (UAT) with a pilot group.

UQ24CA342A Software Testing and Automation

Developing the Test Schedule



Ex: Online food delivery app

3. Determine the order of testing activities

Order for food delivery app:

Unit testing → 2. Integration testing → 3. Functional testing →

UI/UX testing → 5. Performance testing → 6. Security testing → 7. UAT.

This ensures basic functionality is verified before load/security/user testing.

UQ24CA342A Software Testing and Automation

Developing the Test Schedule



Ex: Online food delivery app

4. Allocate resources

Human Resources:

Unit & integration testing → Development team (3 developer)

Functional & UI/UX → 4 QA engineers

Performance → 1 performance tester + 1 developer

Security → Security specialist

UAT → 20 pilot users.

Tools & Infrastructure:

Selenium, JMeter, OWASP ZAP, AWS test server, staging database.

UQ24CA342A Software Testing and Automation

Developing the Test Schedule

Ex: Online food delivery app

5. Develop the test schedule

Week	Activity	Resources
Sept 1–3	Unit testing	Dev team
Sept 4–7	Integration testing	Dev + QA
Sept 8–14	Functional testing	QA team
Sept 15–17	UI/UX testing	QA + Designer
Sept 18–21	Performance testing	Performance tester
Sept 22–25	Security testing	Security engineer
Sept 26–Oct 2	UAT	Stockholders
Oct 3–10	Defect fixes & retesting	Dev + QA

UQ24CA342A Software Testing and Automation

Developing the Test Schedule



Ex: Online food delivery app

6. Review and refine

- QA lead checks if timelines are realistic.
- Security testing moved earlier if it's a regulatory requirement.
- Performance testing extended by 2 days due to complexity.

UQ24CA342A Software Testing and Automation

Developing the Test Schedule



Ex: Online food delivery app

7. Communicate the test schedule

Share the final schedule with:

Dev team

QA team

Product manager

Stakeholders.

Everyone knows who is doing what, and when.

UQ24CA342A Software Testing and Automation

Classroom Activity

The following steps are given in random order. Arrange them in the correct sequence.

1. Allocate Resources

2. Develop the Test Schedule

3. Determine the Testing Timeline

4. Review and Refine

5. Identify Testing Activities

6. Determine the Order of Testing Activities

7. Communicate the Test Schedule

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is a test schedule?

- A) A document containing only test cases
- B) A detailed plan outlining testing activities, timelines, and resources
- C) A software development plan
- D) A defect report

2. Why is it important to determine the order of testing activities?

- A) To reduce the number of testers
- B) To ensure activities are performed in the correct sequence
- C) To increase the project budget
- D) To eliminate testing

3. Why should a test schedule be reviewed and refined?

- A) To increase project cost
- B) To identify improvements and ensure feasibility
- C) To remove testing activities
- D) To replace the testing team

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. B
3. B

A test case is a set of conditions or variables under which a tester determines whether a software system is working correctly or not.

The process of defining test cases involves the following steps:

- Identify the objective of the test case
- Determine the test input data
- Define the expected output
- Specify the steps to be executed
- Document the test case
- Review and refine the test case
- Group related test cases:
- Prioritize the test cases

- 1. Identify the objective of the test case:** Define the purpose of the test case and what specific functionality or behavior is to be tested.
- 2. Determine the test input data:** Decide on the input data that needs to be provided to the software for the test case.
- 3. Define the expected output:** Determine what the expected output of the test case should be, based on the input data and the requirements.
- 4. Specify the steps to be executed:** Define the steps that need to be followed to execute the test case. This should include the setup required to prepare the system for testing, the specific actions to be taken, and the data to be used.
- 5. Document the test case:** Write down the details of the test case, including the objective, input data, expected output, and steps to be executed.

6. Review and refine the test case: Review the test case to ensure that it is complete, accurate, and effective in verifying the software functionality. Make any necessary revisions to improve the test case.

7. Group related test cases: Group similar or related test cases together, to make it easier to manage and execute them.

8. Prioritize the test cases: Assign priorities to the test cases, based on their importance and the likelihood of finding defects.

9. Define the test coverage: Determine the coverage of the test cases by identifying the requirements, features, and areas of the software that the tests are designed to cover.

UQ24CA342A Software Testing and Automation

Defining Test Cases



Ex: Flight Booking System

1. Identify the Objective of the Test Case

Goal: Verify that a user can successfully search for a one-way flight between two valid cities.

2. Determine the Test Input Data

Inputs:

From: Bangalore

To: Delhi

Departure Date: 10-Aug-2025

Travel Type: One-Way

No. of Passengers: 1 Adult

UQ24CA342A Software Testing and Automation

Defining Test Cases



3. Define the Expected Output

Expected Result:

A list of available flights should be displayed with **flight names, departure and arrival timings, ticket prices, and the "Book" option** for each available flight.

UQ24CA342A Software Testing and Automation

Defining Test Cases



4. Specify the Steps to Be Executed Steps:

1. Open the flight booking application.
2. Click on "**Flights**".
3. Select "**One-Way**" as the travel type.
4. Enter "**Bangalore**" as the departure city.
5. Enter "**Delhi**" as the destination city.
6. Select the **departure date: 10-Aug-2025**.
7. Choose **1 Adult** as the number of passengers.
8. Click on the "**Search**" button.
9. Verify that the search request is processed successfully.
10. Check whether the list of available flights is displayed.

UQ24CA342A Software Testing and Automation

Defining Test Cases



5. Document the Test Case

Field	Description
Test Case ID	TC_FLIGHT_001
Title	Verify flight search between two valid cities
Objective	Ensure that flight search results are displayed for valid source and destination cities.
Input Data	From: Bangalore, To: Delhi, Departure Date: 10-Aug-2025, Travel Type: One-Way, Passengers: 1 Adult
Expected Output	A list of available flights is displayed with flight names, timings, prices, and the "Book" option.
Test Steps	Refer to the steps listed in Step 4: Specify the Steps to Be Executed.
Priority	High
Status	To Be Executed

UQ24CA342A Software Testing and Automation

Defining Test Cases



6. Review and refine the test case

- Check for clarity (steps easy to follow).
- Ensure no missing edge cases (invalid inputs, missing dates, same source/destination).
- Update based on peer review feedback.

7. Group Related Test Cases

Grouped Under: Flight Search Module

Group	Test Case IDs
Search Flights	TC_FLIGHT_001, TC_FLIGHT_002
Filter Flights	TC_FLIGHT_003, TC_FLIGHT_004
Sort Flights	TC_FLIGHT_005
Book Flight	TC_FLIGHT_006, TC_FLIGHT_007
Cancel Booking	TC_FLIGHT_008
View Booking History	TC_FLIGHT_009
Payment Module	TC_FLIGHT_010, TC_FLIGHT_011
User Profile	TC_FLIGHT_012
Notifications	TC_FLIGHT_013
Refund Processing	TC_FLIGHT_014

UQ24CA342A Software Testing and Automation

Defining Test Cases



8. Prioritize the Test Cases

Priority	Reason	Example Test Cases
High	Core functionality that directly impacts flight booking and user experience. These test cases must be executed first.	Search Flights, Book Flight, Payment Processing, User Login
Medium	Important features that enhance usability but do not stop the main booking process if they fail.	Filter Flights, Sort Flights, View Booking History, Notifications
Low	Cosmetic or non-critical features that have minimal impact on business functionality.	UI Theme/Color Changes, Font Size, Layout Adjustments, Animations

Apply all Defining test case steps for the below scenario.

You are working as a **Software Test Engineer** for an **Online Movie Ticket Booking System**.

The application provides the following features:

- User Login
- Search Movies
- Select Seats
- Make Payment
- Download Ticket

In software development, a test case is a set of detailed instructions to test the software application ensuring it works properly for end-users. A test case includes test steps, testing data (such as login details,) an expected result upon successful execution, as well as unexpected results if there are failures.

General structure of Test Case

Test Case ID	Test Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-01						
TC-02						

UQ24CA342A Software Testing and Automation

Test cases

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-01	Verify user login with valid credentials	1. Open Login Page 2. Enter valid username & password 3. Click Login	Username: abc123 Password: Pass@123	User should successfully login and Home page should be displayed.	Home page displayed successfully.	Pass
TC-02	Verify login with invalid password	1. Open Login Page 2. Enter valid username and invalid password 3. Click Login	Username: abc123 Password: abc123	Error message "Invalid Username or Password" should appear.	Error message displayed correctly.	Pass

Open PESU academy in your mobile phone or laptop and write test cases for the following features

- i) Login with wrong SRN or password
- ii) Open unit1 slides
- iii) Check Attendance status

Test Case ID	Test Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-01						
TC-02						

Identifying test data is the process of selecting and preparing data that will be used to test the software.

Test data includes all input values, configuration settings, and other necessary data needed to perform testing on the system.

It will be used to validate the functionality, performance, and security of the software.

There are different types of test data,

- Normal data
- Boundary values
- Negative data
- Error messages.

- **Normal data** is used to test the software under typical operating conditions
- **Boundary values** are used to test the limits of the software.
- **Negative data** is used to test the software's ability to handle invalid input,
- **Error messages** are used to test the software's response to errors.

Example: Flight booking system for a one-way trip from Bangalore

Types of test data

1. Normal Data

Data that a typical user would enter — should lead to a successful booking.

Example:

- From: Bangalore (BLR)
- To: Delhi (DEL)
- Date: 15-Aug-2026
- Passengers: 1 adult
- Payment: Valid Credit Card

Expected Result: Booking confirmation generated with PNR and email notification.

2. Boundary Values

Test limits of input fields to ensure software handles them correctly.

Example:

- Min Passenger Count: 1 adult
- Max Passenger Count: 9 adults (system limit)
- Earliest booking date: Today's date
- Latest booking date: 365 days from today (system limit)

Expected Result:

Accepts min and max passenger/date values.
Rejects booking if passengers > 9 or date beyond 365 days.

3. Negative Data

Invalid values that the system should reject.

Example:

- From: Bangalore (BLR)
- To: Bangalore (BLR) (same source & destination)
- Date: 01-Jan-2000 (past date)
- Passenger count: 0
- Payment: Expired Credit Card

Expected Result: Appropriate validation error messages, no booking created.

4. Error Messages

Specific outputs when invalid or missing data is entered.

Example:

- Leaving “From” or “To” field blank → “Please enter departure and destination.”
- Invalid airport code → “Invalid location code.”
- Past travel date → “Travel date cannot be in the past.”
- Payment declined → “Transaction failed. Please try again.”
- Expected Result: Clear, user-friendly messages that guide correction.

To identify test data, testers need to analyze the following

- Software requirements
- Different scenarios and use cases
- Expected results
- How the software should behave under different conditions.

Testers may use tools such as data generators and test data management tools to generate or manage test data.

Analyze before identifying test data

Software requirements:

- Must allow booking one-way flights between any two cities.
- Should validate date formats, airport codes, passenger limits, payment details.
- Should display correct search results and booking confirmation.

Different scenarios & use cases:

- Valid booking with correct details.
- Invalid date or location entries.
- Boundary passenger numbers.
- Special offers, discounts.
- Handling unavailable flights.

UQ24CA342A Software Testing and Automation

Identifying Test Data



Expected results:

- Booking confirmation generated for valid data.
- Proper error messages for invalid inputs.
- Payment processed only for valid transactions.

How software behaves under conditions:

- Handles high passenger count gracefully.
- Rejects invalid routes.
- Displays “No flights available” when applicable.

Data generators

A banking application requires testing with 5,000 new customers. Instead of manually entering customer details, a data generator creates 5,000 new customers data

Ex:

Customer ID	Name	Email	Phone	Age
C001	Rahul Sharma	rahul@gmail.com	9876543210	25
C002	Priya Nair	priya@gmail.com	9988776655	32
C003...C5000	Arjun Rao	arjun@gmail.com	9123456789	41

Test Data Management tools

- A banking application requires testing with 5,000 new customers. Instead of manually entering customer details, a data generator creates 5,000 new customers data
- Ex: Before using this data in the testing environment

Customer Name	Credit Card Number
Rahul Sharma	4111 1111 1111 1111
Priya Nair	5555 4444 3333 2222

- the TDM tool masks sensitive information

Customer Name	Credit Card Number
Rahul Sharma	XXXX XXXX XXXX 1111
Priya Nair	XXXX XXXX XXXX 2222

When defects are found, they need to be recorded, tracked, and managed to ensure that they are fixed and the software meets the desired level of quality.

steps:

- Defect Logging
- Defect Classification
- Defect Analysis
- Defect Assignment
- Defect Fixing
- Defect Verification
- Defect Closure

UQ24CA342A Software Testing and Automation

Defect Management Process



Defect Logging:

Defects need to be logged in a defect tracking tool or system. The defect should be logged with a unique identifier, a summary of the issue, a detailed description, the steps to reproduce the defect, and the severity of the defect.

Defect classification:

Once the defect is logged, it needs to be classified based on its severity and priority. This helps in prioritizing the defects based on the level of impact they have on the software.

UQ24CA342A Software Testing and Automation

Defect Management Process



Defect analysis:

Once the defects are classified, the next step is to analyze the defects to identify the root cause of the issue. This helps in fixing the issue and preventing it from occurring in the future.

Defect assignment:

After the defect analysis, the defect is assigned to a developer or a team for fixing. The developer or the team should be provided with all the necessary details to reproduce the issue and fix the defect.

UQ24CA342A Software Testing and Automation

Defect Management Process



Defect fixing:

The assigned developer or team will work on fixing the defect. Once the defect is fixed, the developer or the team should update the status of the defect in the defect tracking system.

Defect verification:

After the defect is fixed, it needs to be verified to ensure that the issue has been resolved. The tester or the test team should verify the defect to ensure that it has been fixed and that there are no side effects.

UQ24CA342A Software Testing and Automation

Defect Management Process



Defect closure:

Once the defect has been verified, it can be closed in the defect tracking system. The status of the defect should be updated as “Closed” in the defect tracking system.

Example: Login Page Defect

1. Defect Logging

Observed Issue: Even when entering correct username and password, login fails.

Defect ID: LGN-01

Severity: High

Status: New

2. Defect Classification

Type: Functional defect

Module: Login

Priority: P1 (must be fixed before release)

Example: Login Page Defect

3. Defect Analysis

Developer checks code.

Finds that the password comparison is **case-sensitive** in a place where it should not be (requirement says case-insensitive).

4. Defect Assignment

Assigned to **Backend Developer – Authentication Module**.

5. Defect Fixing

Developer updates code so passwords are compared in the correct case format.

Example: Login Page Defect

6. Defect Verification

Tester tries correct username and password again.
Login works as expected.

7. Defect Closure

Tester changes defect status to **Closed** in the tracking tool with the note:
“Verified — login works with valid credentials.”

1. Which type of test data is primarily used to test the limits of input values?

- A) Normal Data
- B) Boundary Values
- C) Error Data
- D) Invalid Data

2. The main purpose of testing error messages is to verify:

- A) System performance
- B) Software installation
- C) Software response to invalid operations or failures
- D) Database backup

3. During which phase is the root cause of a defect investigated?

- A) Defect Analysis
- B) Defect Assignment
- C) Defect Closure
- D) Defect Logging

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. C
3. A

Stop testing criteria refers to the conditions that need to be met for the testing process to be stopped.

These criteria help determine whether the product or system under test is ready for release or if further testing is required.

Stop testing criteria may include factors:

- Meeting predefined requirements
- Passing a set number of test cases
- Reaching a specific level of test coverage
- Achieving acceptable performance and quality levels.

Stop Testing Criteria

It is important for testers and stakeholders to agree on the stop testing criteria before the testing process begins to avoid confusion and ensure that everyone is working toward the same goal.

- It helps ensure that the test plan is accurate, complete, and meets the requirements of the project.
- The review process should involve stakeholders, including the project team, quality assurance team, and management.
- The review should include a comprehensive analysis of the test plan, including test objectives, scope, approach, resources, schedule, test cases, and test data.
- Once the review is complete, the test plan should be approved by the project sponsor or project manager.
- Approval indicates that the test plan is acceptable, and the testing process can proceed.

- It is important to remember that the test plan is a living document, and it may need to be updated or modified as the project progresses.
- Any changes to the test plan should be reviewed and approved by the appropriate stakeholders.

The following are the reasons why test planning is important:

- Provides a clear road map
- Minimizes risk
- Reduces costs
- Improves test coverage
- Increases test efficiency
- Provides a basis for review and approval

Provides a clear road map:

- Test planning provides a clear roadmap for the testing phase of the SDLC.
- It outlines the testing objectives, scope, approach, resources, and schedule for testing.
- This road map helps to ensure that testing is done in a structured and organized manner, which helps to minimize the risk of defects in the software application.

Minimizes risk:

- Test planning helps to minimize the risk of defects in the software application.
- By defining a comprehensive set of test cases and identifying the required resources, test planning ensures that all aspects of the software application are tested thoroughly.

Reduces costs:

- Test planning helps to reduce the costs of testing.
- By identifying the required resources and developing a realistic and achievable test schedule, test planning helps to ensure that testing is done efficiently and cost-effectively.

Improves test coverage:

- Test planning helps to improve test coverage by ensuring that all features and functionalities of the software application are tested.
- This helps to ensure that the software application meets the required quality standards.

Increases test efficiency:

- Test planning increases test efficiency by defining a testing approach that is appropriate for the testing objectives, scope, and available resources.
- This ensures that testing is done in a manner that is both effective and efficient.

Provides a basis for review and approval:

- Test planning provides a basis for review and approval of the testing phase of the SDLC.
- It ensures that all stakeholders are aware of the testing objectives, scope, approach, resources, and schedule, and have an opportunity to provide input and feedback.

UQ24CA342A Software Testing and Automation

Test Plan Document

Creating a test plan document is an essential part of the test planning process, and it provides a detailed road map for the testing process.

The following are usually the contents of a test plan document:

1. Introduction	8. Test automation
2. Testing objectives	9. Risks and issues
3. Testing approach	10. Reporting and communication
4. Testing schedule	11. Conclusion
5. Test environment	
6. Test data	
7. Test cases	

UQ24CA342A Software Testing and Automation

Test Plan Document example

Section	Example
1. Introduction	This test plan describes the testing activities for the Online Shopping Application. The goal is to ensure that all major features work correctly before deployment.
2. Testing Objectives	• Verify all functional requirements. • Identify and report defects. • Ensure the application is stable and reliable before release.
3. Testing Approach	Functional Testing, Integration Testing, System Testing, Regression Testing, and User Acceptance Testing (UAT) will be performed. Both manual and automated testing will be used.
4. Testing Schedule	Day 1: Test Planning Day 2–3: Test Case Preparation Day 4–6: Test Execution Day 7: Bug Reporting Day 8: Regression Testing Day 9: Final Test Report
5. Test Environment	Operating System: Windows 11 Browser: Google Chrome, Microsoft Edge Database: MySQLApplication Server: Apache Tomcat

UQ24CA342A Software Testing and Automation

Test Plan Document example

Section	Example
6. Test Data	Valid login credentials, invalid usernames/passwords, sample products, payment details, customer addresses, boundary values (e.g., quantity 1 and 100).
7. Test Cases	TC-01: Verify Login TC-02: Search Product TC-03: Add to Cart TC-04: Remove from Cart TC-05: Successful Payment TC-06: Order Confirmation
8. Test Automation	Selenium WebDriver will automate Login, Search, Add to Cart, and Checkout. Regression test cases will be executed automatically after every build.
9. Risks and Issues	• Delay in software delivery. • Test environment unavailable. • Requirement changes during testing. • Third-party payment gateway downtime.

UQ24CA342A Software Testing and Automation

Test Plan Document example

Section	Example
10. Reporting and Communication	Daily testing status reports will be shared with the Project Manager. Defects will be tracked using Jira. Weekly meetings will review testing progress.
11. Conclusion	Testing is considered complete when all planned test cases are executed, critical defects are fixed, regression testing passes, and the Test Summary Report is approved.



UQ24CA342A Software Testing and Automation

Test Planning Tools and Techniques



Helps to testing teams to create a comprehensive test plan.

The following are some of these tools and techniques:

- Risk analysis
- Test estimation
- Test case design techniques
- Test management tools
- Test automation tools
- Traceability matrix
- Gantt chart

UQ24CA342A Software Testing and Automation

Test Planning Tools and Techniques



Risk analysis: Risk analysis is a technique used to identify and analyze potential risks that may occur during the testing process.

Risk analysis helps to prioritize and allocate resources effectively to manage risks.

There are several methods for performing risk analysis, such as

- Failure Modes and Effects Analysis (FMEA)
- Strengths
- Weaknesses
- Opportunities,
- Threats (SWOT)
- Political, Economic, Social, Technological, Environmental, and Legal (PESTEL).

Test estimation:

- Test estimation is a technique used to estimate the time and effort required to complete the testing process.
- There are several methods for test estimation, such as expert judgment, historical data, and bottom-up and top-down estimation.

Test case design techniques:

- Test case design techniques are used to create test cases that cover all possible scenarios and use cases of the software application being tested.
- Some of the popular test case design techniques are boundary value analysis, equivalence partitioning, and decision table testing.

UQ24CA342A Software Testing and Automation

Test Planning Tools and Techniques



Test management tools:

- Test management tools are used to manage the testing process, including test planning, test case creation, execution, and reporting.
- Some popular test management tools are Jira, HP ALM, and TestRail.

Test automation tools:

- Test automation tools are used to automate the testing process and improve the efficiency and accuracy of the testing.
- Some popular test automation tools are Selenium, Appium, and TestComplete.

UQ24CA342A Software Testing and Automation

Test Planning Tools and Techniques



Traceability matrix:

- A traceability matrix is a tool used to track the relationship between requirements and test cases.
- It helps to ensure that all the requirements are covered by the test cases and provides visibility into the testing process.

Gantt chart:

- A Gantt chart is a tool used to represent the testing schedule in a graphical format.
- It helps to visualize the testing timeline and identify any potential schedule conflicts.

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. Which of the following is NOT typically included in a Test Plan?

- A) Test Objectives
- B) Test Environment
- C) Source Code
- D) Test Schedule

2. The Test Environment section of a Test Plan specifies:

- A) Programming language used
- B) Hardware, software, operating system, browsers, and database required for testing
- C) Customer requirements
- D) Project budget

3. What is the main purpose of a Traceability Matrix (RTM)?

- A) To monitor CPU usage
- B) To map requirements with test cases
- C) To automate test execution
- D) To create UML diagrams

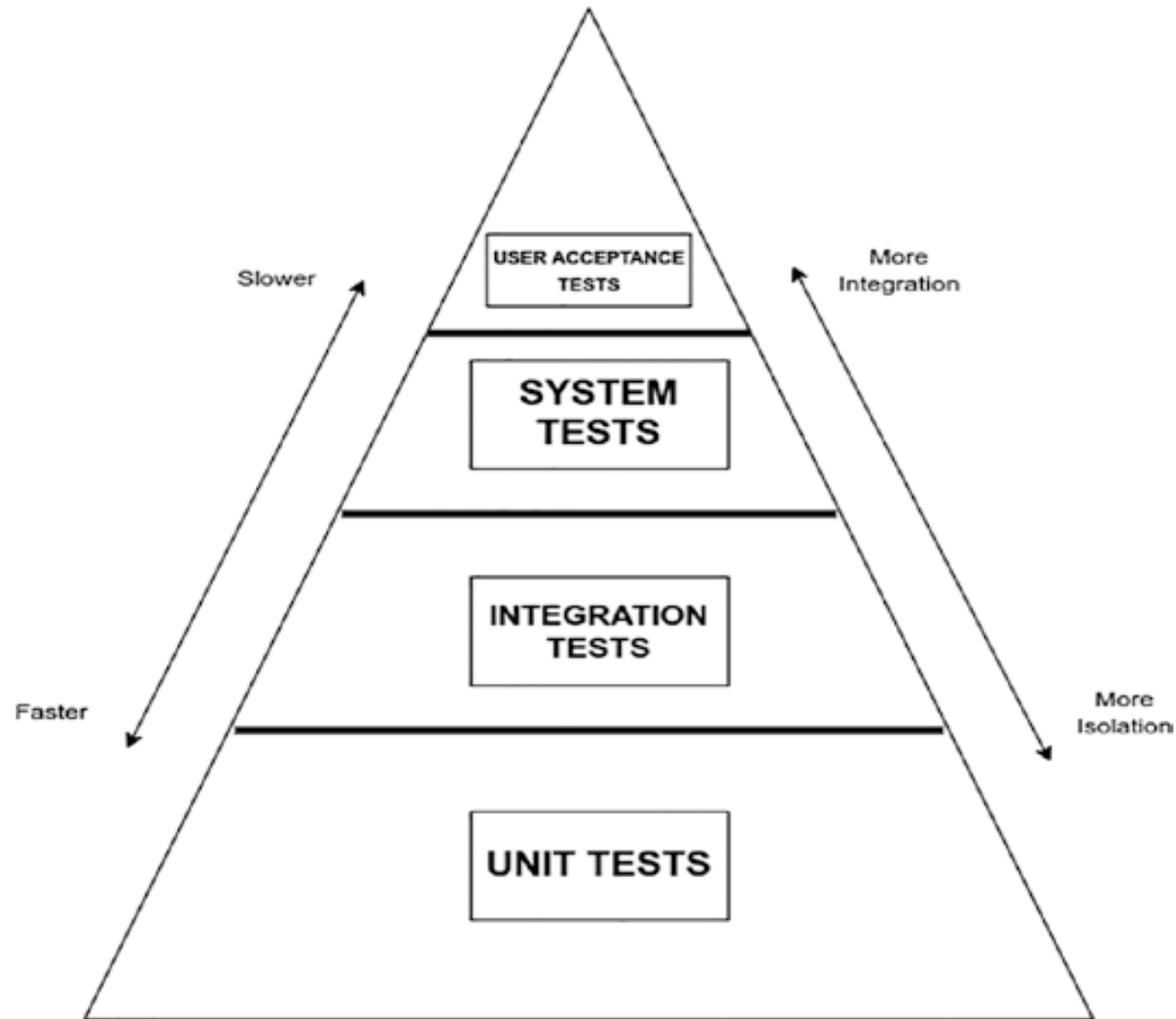
UQ24CA342A Software Testing and Automation

Classroom Quiz

1. C
2. B
3. B

- Validate that the software is working as expected and meets the user's requirements.
- The primary focus is on the software's functionality
- Includes various levels of testing such as unit testing, integration testing, system testing, and acceptance testing

Testing Pyramid

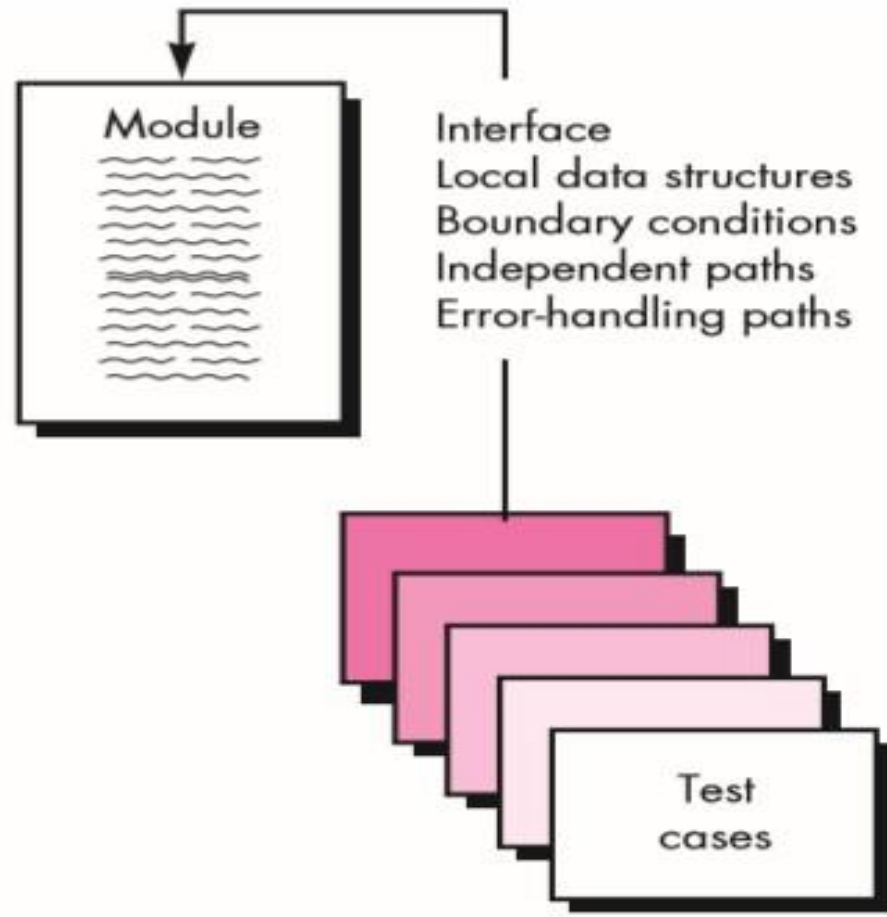


The testing pyramid

- Unit testing is a software testing technique that focuses on testing individual units or components of a software system.
- A unit is the smallest testable part of any software application, usually a function, method, or class.
- The purposes of unit testing are to verify that each unit of the software performs as expected and to catch errors as early as possible in the development process.
- During unit testing, the individual units of the software are tested in isolation from the rest of the system.
- This is achieved by using stubs and Drivers
- Unit tests are typically written by developers as part of the development process

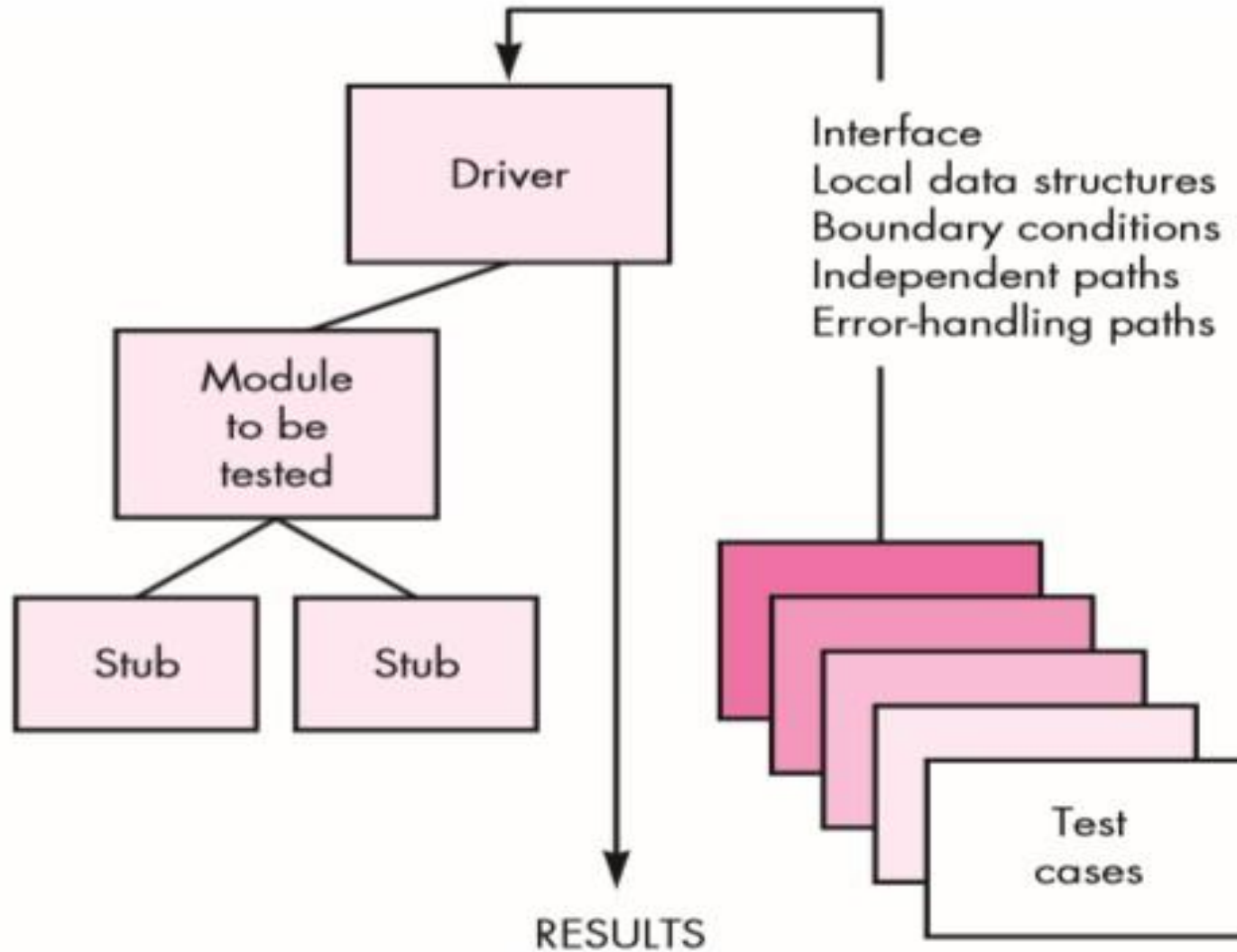
UQ24CA342A Software Testing and Automation

Unit Testing...



UQ24CA342A Software Testing and Automation

Unit Test Environment



Let's use a car as a real-time example to understand Unit Testing, Stub, and Driver.

1. Unit Testing (Testing a single component in isolation)

A car has many components: **engine, brakes, steering, transmission, fuel system, etc.**

In **unit testing**, we test **one small component** separately, without depending on the whole car.

Example:

Suppose we want to test the **car's brake system**.

Instead of driving the whole car, we simulate conditions and test only whether the brake applies the required force, if ABC works, etc.

The test ensures the **brake unit** works correctly, independent of the engine, steering, or other systems.

So, **Unit Testing** = testing **individual car parts** (like brake, engine control module, fuel injector) **separately**.

2. Stub (Temporary dummy for missing dependency that is called by the unit)

A **stub** is a fake module that **provides data to the unit under test** when the real dependency isn't available.

Example:

Suppose we are testing the **car's fuel gauge (Unit Under Test)**.

The gauge normally reads from the **fuel tank sensor**.

If the real sensor is not ready or difficult to use, we create a **stub sensor** that just sends fake values like *“full”, “half”, “empty”*.

The **stub** helps us test the gauge without needing the actual fuel tank sensor.

So, **Stub = Fake supplier of inputs** to the unit.

3. Driver (Temporary dummy that calls the unit under test)

A **driver** is a fake module that **calls/invokes the unit under test**, when the higher-level module (real caller) is not available yet.

Example:

Suppose we are testing the **engine control module (ECM)**.

Normally, the ECM is triggered by the **car ignition system**.

If the ignition system is not ready, we create a **driver** that simulates turning the key/start button, and calls the ECM to start.

The **driver** is like a test harness that initiates the unit.

So, **Driver = Fake caller that triggers the unit.**

Some key characteristics of unit tests:

Isolation: Unit tests are designed to test individual units of code in isolation from the rest of the application. This is typically achieved by using mock objects or stubs to simulate the behavior of other parts of the application.

Automation: Unit tests are typically automated using testing frameworks such as JUnit or NUnit. This allows developers to run tests frequently and quickly and to identify and fix defects early in the development process.

Coverage: Unit tests should provide coverage for all possible execution paths through the unit of code being tested. This helps to ensure that all possible scenarios are tested and that defects are identified early in the development process.

Some key characteristics of unit tests:

Independence: Unit tests should be independent of each other and should not rely on the results of other tests. This helps to ensure that defects are identified and fixed in a timely manner and that tests do not interfere with each other.

Maintainability: Unit tests should be easy to maintain and update as the code evolves over time. This can help to ensure that tests continue to provide accurate results and that defects are identified and fixed early in the development process.

Early detection of bugs: By testing individual units in isolation, unit testing can detect errors early in the development process, which makes them easier and cheaper to fix.

Faster debugging: Unit testing allows developers to isolate and identify the cause of errors quickly and efficiently.

Better code quality: Unit testing forces developers to write modular, well-structured code that is easier to understand and maintain.

Improved design: Unit testing encourages developers to design their code with testability in mind, which can lead to better software design.

- Integration testing is a type of testing that focuses on testing the interaction between different software components, subsystems, or modules.
- The purpose of integration testing is to verify that the integrated software system works as intended and that the individual components are able to work together without errors or issues.
- The main goal of integration testing is to identify any defects or issues that may arise when multiple software components are combined.

UQ24CA342A Software Testing and Automation

Car Example – Fuel Gauge System



Integration Testing Stage:

- Now we **remove the stub and driver**, and connect the **real sensor + real ignition system** with the fuel gauge.
- We check if the **sensor correctly sends signals** to the gauge, and the **gauge displays fuel levels properly when ignition turns on**.

Checking the following:

- You connect the **fuel tank sensor** to the **gauge** and turn the **ignition key**.
- If the tank is half full, does the gauge needle show *half*?
- If the ignition is off, does the gauge stay inactive?
- If the sensor sends faulty values, does the gauge still behave consistently?

Component-level integration: This involves testing the integration of individual components or modules of an application. Component-level integration tests ensure that individual components work together correctly and can communicate with each other.

System-level integration: This involves testing the integration of multiple components or subsystems of an application. System-level integration tests ensure that the application functions correctly as a whole and that different subsystems work together seamlessly.

End-to-end integration: This involves testing the integration of the entire application, including all subsystems and external systems that the application interacts with. End-to-end integration tests ensure that the application meets the needs of end users and stakeholders and that all the components work together correctly.

UQ24CA342A Software Testing and Automation

Example for various levels of Integration



Online Food Delivery App Example

The app has these main parts:

Restaurant Module (menu, pricing, availability)

Cart & Payment Module (add to cart, checkout, payment gateway)

Delivery Module (delivery partner assignment, tracking)

Notification Module (SMS/Push/email updates)

Component-level Integration

Inside the **Restaurant Module**, the **Menu Component** shows dishes and the **Availability Component** tells if a dish is in stock.

In **component-level integration testing**, we test:

- If a dish is marked “out of stock”, does the menu correctly disable the "Add to Cart" button?
- If the price changes, does the menu update automatically?

Ensuring that **sub-components of a single module** integrate properly.

System-level Integration

Connect **Restaurant Module + Cart Module + Payment Module**.

Test flow:

- User selects a dish from the restaurant menu.
- The dish goes into the **cart**.
- At checkout, the **payment gateway** is triggered.

Check if **data flows correctly** (dish details, price, discount, payment status).

Ensuring that **major modules of the food delivery system** integrate and exchange data correctly.

UQ24CA342A Software Testing and Automation

Example for various levels of Integration



End-to-End Integration

Full flow:

- User opens the app and selects a restaurant (Restaurant Module).
- User adds food to cart and pays online (Cart & Payment Module + Payment Gateway).
- Order is sent to the restaurant's system (External Vendor System).
- A delivery partner is assigned (Delivery Module).
- User gets SMS/email/push notification (Notification Module).
- User tracks the delivery in real time until the food arrives.

Ensuring that **the complete ecosystem** (App + Payment Gateway + Restaurant + Delivery + Notifications) works **seamlessly from start to finish**.

Big Bang integration testing: In this type of testing, all the software components are integrated together and tested as a whole.

Incremental integration testing: In this type of testing, the software components are integrated incrementally, one at a time, and tested as they are added to the system.

Top-down integration testing: In this type of testing, testing starts with the highest level modules and works its way down to the lowest level modules.

Bottom-up integration testing: In this type of testing, testing starts with the lowest-level modules and works its way up to the highest-level modules.

Sandpit integration testing: In this type of testing, a new module is integrated with the rest of the system in a separate environment called a sandbox or sandpit. This allows the new module to be tested in a controlled environment without affecting the rest of the system.

Big Bang integration testing:

Modules: Student Registration, Fee Payment, Exams, Reports.

In Big Bang: You build all four modules completely and then integrate them at once.

Incremental integration testing:

First integrate Student Registration + Fee Payment → test.

Then add Exams → test.

Finally add Reports → test.

This way, bugs are found earlier and isolated easily.

Top-down integration testing:

- Start with **ATM Main Menu** (top-level).
- Submenus (Withdraw, Deposit, Balance Enquiry) may not be ready, so you use **stubs** to simulate them.
- As each submenu is developed, replace the stub with the real module.

Bottom-up integration testing:

- Start with **Cash Dispenser** and **Receipt Printer** (lowest modules).
- Use a **driver** to simulate calls.
- Then integrate with **Transaction Processing** module.
- Finally, integrate with **ATM Main Menu**.

Sandpit integration testing:

Mix of **Top-down** and **Bottom-up** approaches. Test critical modules first (both top & bottom), then fill the middle.

Example (E-commerce Website):

- Test **Shopping Cart (middle module)** with **Payment Gateway (bottom module)** and **User Dashboard (top module)** simultaneously.
- Gradually integrate other modules like **Wishlist** or **Coupons** later.

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the primary objective of Unit Testing?

- A) Test the entire software system
- B) Test individual modules or functions independently
- C) Test user acceptance
- D) Test software performance

2. The primary goal of Integration Testing is to verify:

- A) Individual functions
- B) Interaction between integrated modules
- C) User interface design
- D) Database backup

3. Which of the following is NOT an Integration Testing approach?

- A) Top-Down
- B) Bottom-Up
- C) Big Bang
- D) Black Box

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. B
3. D

System testing is a type of software testing that is performed on a complete, integrated system to evaluate the system's compliance with its specified requirements.

Ex: ATM Withdrawal

- Requirement "The ATM should allow cash withdrawal when the account has sufficient balance."
- "Validation Activity"
 - ✓ Insert ATM card.
 - ✓ Enter the correct PIN.
 - ✓ Select Withdraw ₹2,000.Click Confirm.
- Expected Result
 - ✓ ₹2,000 is dispensed.
 - ✓ Account balance is reduced by ₹2,000.
 - ✓ Receipt is printed (if selected).
- Actual Result: All expected actions occur successfully.

- System testing is a type of software testing that is performed on a complete, integrated system to evaluate the system's compliance with its specified requirements.
- It is generally a black-box testing approach, which means that testers are not concerned with the internal workings of the system.
- Instead, they focus on testing the system as a whole to ensure that it meets the functional and nonfunctional requirements of the stakeholders.
- System testing is typically conducted after integration testing

System Testing Example

The Online Shopping Application contains the following modules:

- Login
- Product Search
- Shopping Cart
- Payment
- Order Confirmation

System Testing Activity:

The tester verifies that:

- Login works correctly.
- Products can be searched.
- Items are added to the cart.
- Payment is successful.
- Order confirmation is generated.
- Email notification is sent.

The entire integrated application works correctly

UQ24CA342A Software Testing and Automation

System Testing



Imagine buying a new car.	
Validation "Does this car satisfy my needs?"	System Testing
Is it comfortable? Does it have enough seats? Is it fuel-efficient?	The manufacturer checks: Engine Brakes Steering Airbags Headlights Gearbox
If yes, the product is validated.	They verify that all parts work together before delivering the car.
Validation = Customer's perspective System Testing = Tester's perspective	

Validation	System Testing
Confirms that the software meets the user's needs and requirements.	Tests the entire integrated system as a whole.
Answers the question: "Are we building the right product?"	Ensures that all modules work together correctly.
Focuses on whether the product satisfies business requirements.	Focuses on the functionality, performance, security, and behavior of the complete system.
Can occur through activities such as System Testing, User Acceptance Testing (UAT), or Beta Testing.	Is a specific testing level performed after Integration Testing.
Concerned with customer expectations.	Concerned with the correctness of the complete application.

- User acceptance testing (UAT) is a process to validate whether a system meets the specified requirements and works as expected in the real-world scenario.
- UAT is performed by end users, business stakeholders, or domain experts to ensure the system's functionality and usability.
- Example : Test drive of car done by customer

UQ24CA342A Software Testing and Automation

Primary objectives of UAT

- To validate the system's functionality from an end user's perspective
- To check whether the system meets the business requirements
- To ensure that the system is user-friendly and easy to use
- To find defects that were not detected in previous testing stages
- To reduce the risk of system failure in production

UQ24CA342A Software Testing and Automation

some key characteristics of user acceptance testing



User involvement: UAT involves end users or stakeholders testing the software product, providing feedback, and ensuring that it meets their needs and requirements.

Real-world testing: UAT involves testing the software product in a real-world environment, using real data and scenarios, to ensure that it meets the user's business processes and workflows.

Validation of business requirements: UAT ensures that the software product meets the business requirements and goals of the user or stakeholder.

Acceptance criteria: UAT defines acceptance criteria for the software product, which are used to determine whether the product is ready for deployment.

Sign-off: UAT requires the sign-off of the user or stakeholder to indicate that they are satisfied with the software product and that it is ready for deployment.

There are three types of UAT:

- **Alpha testing:** It is performed in-house by the development team or a group of testers before releasing the product to the end users.
- **Beta testing:** It is performed by a group of end users in a real-world scenario before releasing the product to the market.
- **Acceptance testing:** It is performed by the end users or business stakeholders to validate whether the system meets the specified requirements and works as expected in the real-world scenario.

College Management System like PESU App

Alpha Testing → Tested by the company's QA team before it is released. They check login, attendance, marks entry, and report generation.

Beta Testing → Three colleagues assigned to test for two weeks. Teachers and students use it and report issues like slow performance or confusing screens.

UAT → The college principal and staff verify that the system satisfies all agreed requirements, such as student registration, attendance, fee payment, and report generation.

Acceptance Testing → A college checks whether the College Management System meets all requested features before accepting it.

UQ24CA342A Software Testing and Automation

Nonfunctional Testing



This type of testing focuses on the software's nonfunctional aspects, such as performance, security, and usability.

The objective is to ensure that the software meets its nonfunctional requirements and provides a positive user experience.

Performance testing involves testing the software's performance under various conditions, such as i) load, ii) stress, and iii) scalability.

Security testing involves testing the software's security features to ensure that it is secure from various types of attacks and vulnerabilities.

Usability testing involves testing the software's user interface and user experience to ensure that it is user-friendly and easy to use.

i) Load Testing: Testing how the system performs under normal and expected peak loads.

Ex On a regular sale day, the system is expected to handle **10,000 concurrent users** checking out at the same time. Load testing will check if the checkout process (adding address, payment, order confirmation) works smoothly without delays.

ii) Stress Testing: Pushing the system beyond its maximum load to see how it behaves under extreme conditions.

Ex: During a **flash sale (like Black Friday)**, suddenly **50,000 users** might try to check out at once. Stress testing checks whether the site crashes, slows down, or gracefully denies excess users.

iii) Scalability Testing: Checking if the system can scale up or down when more resources (like servers, bandwidth, or database instances) are added.

Ex: If the website is deployed on cloud servers, scalability testing ensures that when the traffic increases from **10,000 to 30,000 users**, the system automatically adds more servers and continues to perform well.

UQ24CA342A Software Testing and Automation

Security Testing & Usability testing



Security Testing Example: Online Banking Login System

Imagine you are testing the **login feature** of an online banking application. Since this app stores sensitive information (like account details, transactions, personal data), it must be very secure.

Usability testing Example: Food Delivery Mobile App (like Swiggy)

When people order food, they want it to be **quick and easy**. Usability testing ensures that the **user interface (UI)** and **user experience (UX)** are smooth.

Some of the benefits from Test Automation are:

- **Saves time and cost:** Automation allows for the execution of tests at a much faster rate than manual testing, which saves time and reduces the cost of testing.
- **Improves test coverage:** Automation tools can perform repetitive and time-consuming tests, which can improve the overall test coverage.
- **Increases accuracy:** Automation tools can execute tests with greater accuracy than humans, reducing the chances of errors.
- **Reusability:** Test scripts can be reused for different versions of the application, which saves time and effort.
- **Enhances efficiency:** Automation testing can run tests continuously, 24/7, which enhances the efficiency of testing.

Before implementing automation consider the following

- **Cost-benefit analysis:** Determine the cost of automation tools and the benefits it will provide.
- **Technical feasibility:** Ensure that the automation tool is compatible with the technology stack and the application under test.
- **Skillset:** Ensure that the team has the required technical skills to use and maintain the automation tool.
- **Scope:** Determine the scope of automation testing and identify the tests that are good candidates for automation.

UQ24CA342A Software Testing and Automation

Benefits of Test Automation

Types of automation testing

- Record and Playback Tools
- Scripting Tools
- Hybrid Tools
- Frameworks



- The basic process of using a record and playback tool involves starting the recording, performing a series of user actions on the application or system being tested, and then stopping the recording.
- The tool then generates a script based on the recorded actions, which can be modified and enhanced as needed.
- These tools typically capture user actions such as mouse clicks, keyboard inputs, and data entry, and then they generate a script that can be played back to reproduce the same set of actions automatically.
- Record and playback tools are often used for Regression Testing & Smoke Testing

UQ24CA342A Software Testing and Automation

Record and Playback Tools



Limitations

- Not able to handle more complex scenarios or variations in user input
- any changes to the application or system under test can cause the recorded scripts to fail

UQ24CA342A Software Testing and Automation

Record and Playback Tools

Example:



Selenium IDE

Selenium IDE is a browser extension that records user actions and generates test scripts.

Example: Recording a login scenario on a web application.

Selenium IDE - LoginTest*

Project: LoginTest*

Tests

Test Suite

Login Test

https://example.com/login

	Command	Target	Value
1	open	/login	
2	type	id=username	tester
3	type	id=password	password123
4	click	id=loginBtn	
5	verifyText	id=welcome	Welcome

Command

Target

Value

- Scripting tools enable testers to write scripts or code to automate testing tasks.
- These tools allows testers to create custom test cases and automate complex scenarios that may not be possible with record and playback tools.
- Testers need to write the test scripts themselves using languages such as Python, Java, or Ruby, depending on the tool.

Limitations:

- Require a significant amount of time and effort to create and maintain the test scripts.
- Testers need to have programming skills and expertise to write effective and efficient test scripts.

UQ24CA342A Software Testing and Automation

Scripting Tools



Selenium WebDriver

Selenium WebDriver is a scripting tool used to automate web applications using languages like Java, Python, C#, JavaScript.

Example: Login test script using Python.

```
login_test.py x
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3  from selenium.webdriver.common.keys import Keys

4
5  # Initialize WebDriver
6  driver = webdriver.Chrome()

7  # Open application
8  driver.get("https://example.com/login")

9  # Enter username
10 driver.find_element(By.ID, "username").send_keys("tester")

11 # Enter password
12 driver.find_element(By.ID, "password").send_keys("password123")

13 # Click on Login button
14 driver.find_element(By.ID, "loginBtn").click()

15 # Validate login
16 assert "Welcome" in driver.page_source
17
18
19 driver.quit()
```

- Hybrid tools are a type of test automation tool that combine the benefits of both record and playback and scripting tools.
- They provide a user-friendly interface for recording and editing test scripts, while also offering more advanced features for customizing and enhancing scripts as needed.
- This makes them suitable for a wide range of testing scenarios, from simple regression testing to more complex performance testing or load testing.

UQ24CA342A Software Testing and Automation

Frameworks



- Test automation frameworks aim to standardize the testing process and ensure consistency in the way tests are created and executed.
- A test automation framework typically consists of a set of components, such as libraries, APIs, and utilities, that provide the building blocks for creating automated tests.

Types of test automation frameworks

- 1) **Linear scripting framework:** This framework is a basic approach to test automation that involves writing simple scripts that perform a series of predefined actions. It is easy to create and maintain but lacks flexibility and scalability.
- 2) **Modular framework:** This framework breaks down the application or system under test into modules and creates separate scripts for each module. This approach provides greater flexibility and reusability but requires more time and effort to design and implement.

UQ24CA342A Software Testing and Automation

Frameworks



3. **Data-driven framework:** This framework separates the test data from the test script, allowing for more efficient management of large volumes of test data. It can be useful for testing scenarios that require a large number of input combinations.
4. **Keyword-driven framework:** This framework uses a set of keywords or commands to define test steps, making it easier for testers without programming experience to create and execute tests.
5. **Hybrid framework:** This framework combines elements of multiple frameworks, providing greater flexibility and scalability. It is often used for larger, more complex testing projects that require a high level of customization and control..

1. Which testing checks how well an application protects data from unauthorized access?

- A) Functional Testing
- B) Regression Testing
- C) Security Testing
- D) Unit Testing

2. Which of the following is a scripting-based automation tool?

- A) Selenium WebDriver
- B) Paint
- C) MS Word
- D) Calculator

3. Which type of non-functional testing measures the response time and speed of an application?

- A) Functional Testing
- B) Performance Testing
- C) Unit Testing
- D) Integration Testing

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. C
2. A
3. B

Consider the following python programs and perform the following:

- i) Identify the error**
- ii) Correct the code**
- iii) Update the test cases fail status with pass status**

UQ24CA342A Software Testing and Automation

EL on Python Programs

Prerequisites:

- i) Open Python installed on the system
- ii) Open Online GDB “<https://www.onlinegdb.com/>”



Python program to find whether the number is odd or even

```
def check_number(n):  
    if n % 2 == 0:  
        return "Even"  
    else:  
        return "Odd"  
print(check_number(10))
```

UQ24CA342A Software Testing and Automation

EL on Python Programs

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-01	Verify whether the number is even or odd	1. Enter numbers 2. Execute & check the output	10	Even	Even	Pass
TC-02a	Verify whether the number is even or odd	1. Enter Alphabet 2. Execute & check the output	a	Display "Invalid input"	Name Error	Fail
TC-02b	-- --	-- --	a	Display "Invalid input"	Display "Invalid input"	pass
TC-03	Verify whether the number is even or odd	1. Enter number with in double quotes 2. Execute & check the output	"10"			

Corrected code

```
def check_number(n):
```

```
    if n % 2 == 0:
```

```
        print("Even")
```

```
    else:
```

```
        print("Odd")
```

```
x = input("Enter a number: ")
```

```
if x.isdigit():
```

```
    check_number(int(x))
```

```
else:
```

```
    print("Invalid Input")
```

Check for :

```
print(check_number(10))
```

```
print(check_number(7))
```

```
print(check_number("a"))
```

```
print(check_number("hello"))
```

```
print(check_number("20"))
```

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-04	Verify whether the number is even or odd		"20"			
TC-05	Verify whether the number is even or odd		"a"			
TC-06	Verify whether the number is even or odd		"@"			

Python program for login verification

```
def login(username, password):  
    if username == "admin" and password == "1234":  
        return "Login Successful"  
    else:  
        return "Invalid Login"  
  
print(login("admin", "1234"))
```

Corrected code

```
def login(username="", password=""):

    if not username or not password:
        return "Missing username or password"

    if username == "admin" and password == "1234":
        return "Login Successful"
    else:
        return "Invalid Login"
```

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-01	Login verification		Login=admin Password=1234			
TC-02	Login verification		Login=admin Password=1111			
TC-03	Login verification		Login=user Password=1234			

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-04	Login verification		Login=a d m i n Password=1234			
TC-05	Login verification		Login= Password=1234			
TC-06	Login verification		Login=pesuniversity Password=1234			

Python program for ATM amount withdraw

```
def withdraw(balance, amount):  
    if amount <= balance:  
        return balance - amount  
    else:  
        return "Insufficient Balance"  
  
print(withdraw(5000, 1000))
```

Specifically check for balance = 5000 and amount = -500

Python program for Student result

```
def grade(mark):  
    if mark >= 90:  
        return "A"  
    elif mark >= 75:  
        return "B"  
    elif mark >= 50:  
        return "C"  
    else:  
        return "Fail"
```

```
print(grade(85))
```

Specifically check for Marks = 101

Python program for Online Shopping

```
def calculate_price(amount):  
    if amount > 1000:  
        discount = amount * 0.10  
        return amount - discount  
    else:  
        return amount
```

```
print(calculate_price(1100))
```

Specifically check for Amount = 1000

Python program for Voting eligibility

```
def check_age(age):  
    if age >= 18:  
        return "Eligible"  
    else:  
        return "Not Eligible"
```

```
print(check_age(20))
```

Specifically check for Age = -5



THANK YOU

Mrs. Sowmya BP

Department of Computer Applications

sowmyabp@pes.edu